Analysis Report Template

Student Full Name	Koronchige Bhagya Anuhasini Silva	Student ID No.	W2121190/20231820

Case Study Title

Case Study (A): Predicting Clients Loan Status. [Total 43 Marks]

Research Question: Does machine learning have the potential to assist banker and finance analysts in predicting which client can be approved a loan?

Task (1) – Domain Understanding: Classification

The financial risk analysts decided that classification modelling is required. Indicate in the table below fo variables in your data which ones you should RETAIN and can be included in the classification modelling of loan approval stat should DROP (REMOVE). Justify your decision logically and/or by research (for any research, include the Harvard style) **[3 Marks]**

Variable Name	RETAIN or DROP	Brief justification for retention or dropping with in-text citation.
ID	DROP	Just an identifier, it doesn't help predict
Sex	DROP	Gender is not useful for analysis
Age	RETAIN	Older or younger clients may have different risks; numeric and relevant
Education Qualifications	DROP	Education does not reflect on analysis
Income	RETAIN	Higher income often means a better ability to repay loans
Home Ownership	RETAIN	Owning a home reduces lender risk; categorically
Employment Length	RETAIN	Longer employment suggests stability and repayment capacity
Loan Intent	RETAIN	The type of loan may affect risk (Bellotti & Crook, 2009)
Loan Amount	RETAIN	Bigger loans may have higher default risk
Loan Interest Rate	RETAIN	High interest may indicate higher-risk borrowers
Loan-to-Income Ratio (LTI)	RETAIN	Shows ability to repay compared to income; key risk metric (Thomas et al., 2002)
Payment Default on File	RETAIN	Past defaults indicate risk, an important predictor (Bellotti & Crook, 2009)
Credit History Length	RETAIN	A longer history may mean a more reliable borrower
Loan Approval Status	RETAIN	This is the target variable for the classification model we want to predict
Maximum Loan Amount	RETAIN	This is the target variable for the regression model we want to predict

Credit Application Acceptance	DROP	Outcome variable, not an input for prediction

References

Bellotti, T. and Crook, J. (2009) 'Credit scoring and prediction of consumer default', *Journal of the Operational Research Society*, 60(6), pp. 824–834. Available at: https://doi.org/10.1057/palgrave.jors.2602610

Thomas, L.C., Crook, J.N. and Edelman, D.B. (2002) *Credit Scoring and Its Applications*. Philadelphia: SIAM. Available at: https://epubs.siam.org/doi/book/10.1137/1.9780898718317

Task (2) – Exploring and Understanding Your Dataset

With the aid of your Final Python Notebook 1, for your RETAINED input variables and your class "Target" variable, produce 1)**basic descriptive stat**s and 2)**variable scale type, then** 3)**plot the distribution of your target variable**. (Paste the three screenshots of code OUTPUTS ONLY for evidence of these elements).

[2 Marks]

1. **basic descriptive stats**

...

	age	income	emplyment_length	loan_amount	loan_interest_rate	loan_income_ratio	credit_history_length
count	58639.000000	5.864500e+04	58645.000000	58645.000000	58634.000000	58645.000000	58645.000000
mean	27.550913	6.404617e+04	4.703487	9217.556518	10.677526	0.159238	5.813556
std	6.033217	3.793111e+04	4.004982	5563.807384	3.036034	0.091692	4.029196
min	20.000000	4.200000e+03	0.000000	500.000000	-11.140000	0.000000	2.000000
25%	23.000000	4.200000e+04	2.000000	5000.000000	7.880000	0.090000	3.000000
50%	26.000000	5.800000e+04	4.000000	8000.000000	10.750000	0.140000	4.000000
75%	30.000000	7.560000e+04	7.000000	12000.000000	12.990000	0.210000	8.000000
max	123.000000	1.900000e+06	150.000000	35000.000000	23.220000	0.830000	30.000000

Figure 1 Basic descriptive stats for retained variables

	loan_approval_status
count	58645.000000
mean	0.142382
std	0.349445
min	0.000000
25%	0.000000
50%	0.000000
75%	0.000000
max	1.000000

dtype: float64

	max_allowed_loan
count	5.864500e+04
mean	6.975472e+04
std	6.175091e+04
min	-2.426900e+06
25%	3.800300e+04
50%	6.239200e+04
75%	9.271600e+04
max	2.638778e+06

dtype: float64

Figure 2 Basic descriptive stats for the target variables

2. variable scale type

```
age                          float64
income                         int64
home_ownership                object
emplyment_length               int64
loan_intent                   object
loan_amount                    int64
loan_interest_rate           float64
loan_income_ratio            float64
payment_default_on_file       object
credit_history_length          int64
dtype: object
```

Figure 3 variable scale type for retained variables

```
•••  int64
```

Figure 4 variable scale type for target variable

3. Plot the distribution of your target variable

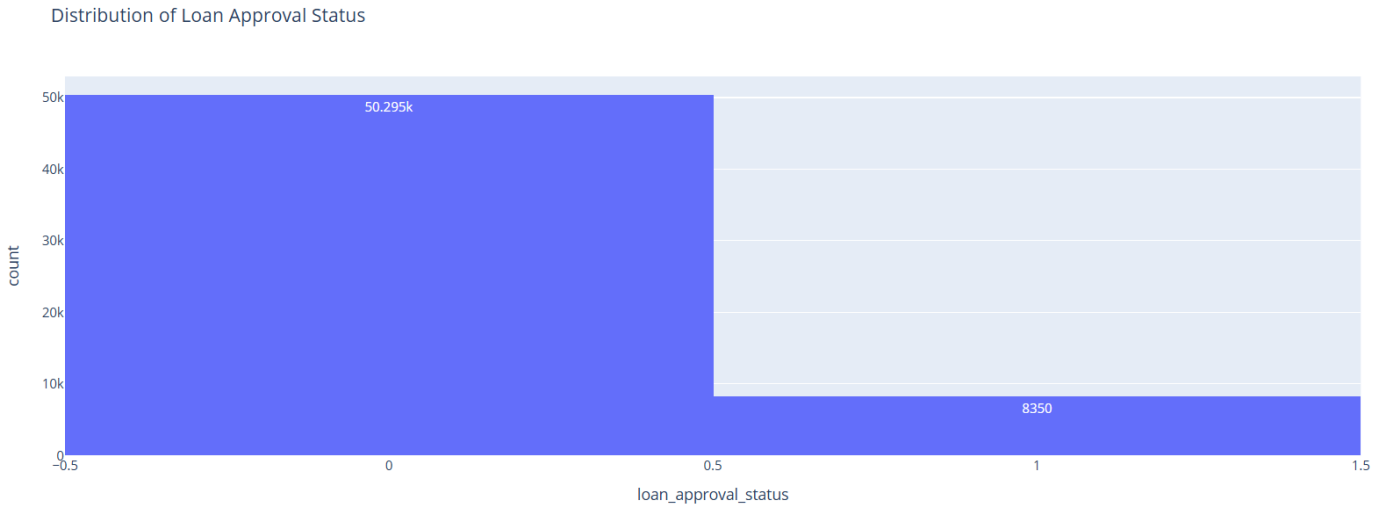


Figure 5 Plot the distribution of your target variable classification

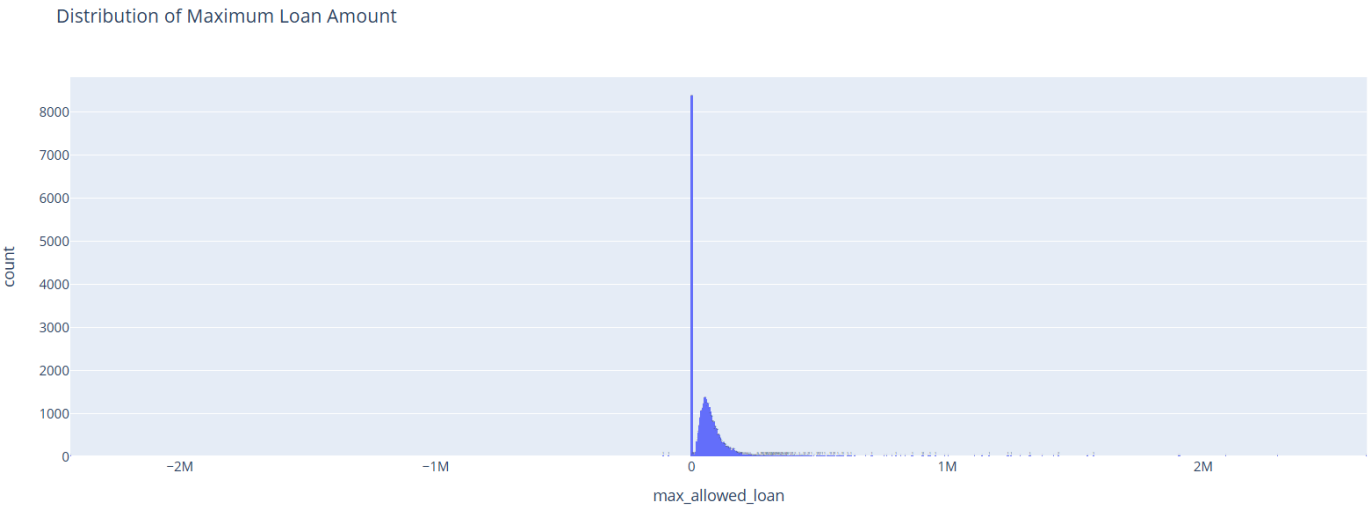


Figure 6 Plot the distribution of your target variable regression

Task (3) – Data Preparation: Cleaning and Transforming your data

a) With the aid of your Final Python Notebook 1, when you first explored your retained variables in the loan dataset, you may have found some issues. **1) Report any issues you found in your retained dataset variables**. Based on the issues you found in your data, **2)suggest a suitable possible method to fix each of these issues** and **3)provide your justification for using your suggested fix method**. Use the table below to organise your findings and analysis, and add more rows if needed

[4 Marks]

	Variable Name	Issue found	Proposed fix	Justification for used fix method
1	id, opposite target variable	Irrelevant/unnecessary variables present in the dataset	Drop unnecessary columns (id and opposite target variable) from both classification and regression datasets	These columns do not contribute to model prediction. id is just an identifier, and the opposite target would cause data leakage
2	home_ownership, loan_intent, payment_default_on_file	Categorical (text) data that ML algorithms cannot process directly	Label Encoding using LabelEncoder() to convert categories to numeric values	ML algorithms perform mathematical computations and require numeric input. Label Encoding converts text categories into integers that the model can interpret
3	age,income, emplyment_length, loan_amount, loan_interest_rate, loan_income_ratio, credit_history_length	Outliers detected in numeric variables using IQR method	Clip outliers to IQR boundaries (lower = Q1 − 1.5×IQR, upper = Q3 + 1.5×IQR)	Outliers can skew model training and reduce accuracy. Clipping retains all rows while limiting extreme values to acceptable boundaries, preserving data size
4	age, income, emplyment_length, loan_amount, loan_interest_rate, loan_income_ratio, credit_history_length	Missing values found in numeric columns	Fill missing values with the **median** of each column	The median is robust to outliers, unlike the mean. It fills gaps without distorting the distribution of skewed numeric data
5	home_ownership, loan_intent, payment_default_on_file	Missing values found in categorical columns	Fill missing values with the **mode** (most frequent value) of each column	Mode is the most appropriate central tendency measure for categorical data, replacing missing entries with the most common category
6	Entire dataset	Search for Duplicate rows can be present in the dataset	No Duplicate rows present in the dataset	Duplicate rows can bias the model by over-representing certain patterns, leading to overfitting and inaccurate performance metrics

Task (3) – Data Preparation: Cleaning and Transforming your data

b) With the aid of Python packages and your Final Python Notebook 1, implement your suggested fixes of issues in the previous Task 3-a in your final Python Notebook1.

1) Show evidence (before and after) of implementing your suggested fix to the problems you identified for your dataset in Task 3-a.

To show your evidence, paste screenshots of your relevant code OUTPUTS ONLY (Do not paste the code).

2) Indicate and annotate the issue and the fix in each of your provided evidence screenshots.

[4 Marks]

Output screenshot of the issue before the fix	**Output screenshot after fixing the issue**

	id	age	income	emplyment_length	loan_amount	loan_interest_rate	loan_income_ratio	credit_history_length	loan_approval_status	max_allowed_loan
count	58645.000000	58639.000000	5.864500e+04	58645.000000	58645.000000	58634.000000	58645.000000	58645.000000	58645.000000	5.864500e+04
mean	29322.000000	27.550913	6.404617e+04	4.703487	9217.556518	10.677526	0.159238	5.813556	0.142382	6.975472e+04
std	16929.497605	6.033217	3.793111e+04	4.004982	5563.807384	3.036034	0.091692	4.029196	0.349445	6.175091e+04
min	0.000000	20.000000	4.200000e+03	0.000000	500.000000	-11.140000	0.000000	2.000000	0.000000	-2.426900e+06
25%	14661.000000	23.000000	4.200000e+04	2.000000	5000.000000	7.880000	0.090000	3.000000	0.000000	3.800300e+04
50%	29322.000000	26.000000	5.800000e+04	4.000000	8000.000000	10.750000	0.140000	4.000000	0.000000	6.239200e+04
75%	43983.000000	30.000000	7.560000e+04	7.000000	12000.000000	12.990000	0.210000	8.000000	0.000000	9.271600e+04
max	58644.000000	123.000000	1.900000e+06	150.000000	35000.000000	23.220000	0.830000	30.000000	1.000000	2.638778e+06

Figure 7 Before dropping unnecessary variables

	age	income	home_ownership	emplyment_length	loan_intent	loan_amount	loan_interest_rate	loan_income_ratio	payment_default_on_file	credit_history_length	loan_approval_status
0	21.0	12000	OWN	0	EDUCATION	15000	6.99	0.12	N	4	0
1	21.0	13200	OWN	2	EDUCATION	25000	16.77	0.19	Y	3	0
2	23.0	9600	RENT	5	MEDICAL	30000	12.42	0.31	N	3	0
3	40.0	182004	RENT	3	EDUCATION	35000	8.00	0.19	N	11	0
4	40.0	90000	MORTGAGE	3	HOMEIMPROVEMENT	35000	12.42	0.39	N	14	0

Figure 8 After dropping unnecessary variables for the classification model

	age	income	home_ownership	emplyment_length	loan_intent	loan_amount	loan_interest_rate	loan_income_ratio	payment_default_on_file	credit_history_length	max_allowed_loan
0	21.0	12000	OWN	0	EDUCATION	15000	6.99	0.12	N	4	-2426900
1	21.0	13200	OWN	2	EDUCATION	25000	16.77	0.19	Y	3	-111739
2	23.0	9600	RENT	5	MEDICAL	30000	12.42	0.31	N	3	-69000
3	40.0	182004	RENT	3	EDUCATION	35000	8.00	0.19	N	11	35000
4	40.0	90000	MORTGAGE	3	HOMEIMPROVEMENT	35000	12.42	0.39	N	14	35000

Figure 9 After dropping unnecessary variables for the regression model

	age	income	home_ownership	emplyment_length	loan_intent	loan_amount	loan_interest_rate	loan_income_ratio	payment_default_on_file	credit_history_length	loan_approval_status
0	-1.085843	-1.372135	OWN	-1.174419	EDUCATION	1.039305	-1.214715	-0.427932	N	-0.450108	0
1	-1.085843	-1.340499	OWN	-0.675037	EDUCATION	2.836851	2.006922	0.335502	Y	-0.698298	0
2	-0.754326	-1.435409	RENT	0.074037	MEDICAL	3.735324	0.573985	1.644245	N	-0.698298	0
3	2.063567	3.109818	RENT	-0.425346	EDUCATION	4.633996	-0.882010	0.335502	N	1.287227	0
4	2.063567	0.684242	MORTGAGE	-0.425346	HOMEIMPROVEMENT	4.633996	0.573985	2.516741	N	2.031798	0

Figure 10 Before Label Encoding

	age	income	home_ownership	emplyment_length	loan_intent	loan_amount	loan_interest_rate	loan_income_ratio	payment_default_on_file	credit_history_length	loan_approval_status
0	-1.085843	-1.372135	2	-1.174419	1	1.039305	-1.214715	-0.427932	0	-0.450108	0
1	-1.085843	-1.340499	2	-0.675037	1	2.836651	2.006922	0.335502	1	-0.698298	0
2	-0.754326	-1.435409	3	0.074037	3	3.735324	0.573985	1.644245	0	-0.698298	0
3	2.063567	3.109818	3	-0.425346	1	4.633996	-0.882010	0.335502	0	1.287227	0
4	2.063567	0.684242	0	-0.425346	2	4.633996	0.573985	2.516741	0	2.031798	0

Figure 11 After Label Encoding

```
Outlier counts BEFORE clipping:

age: 2446
income: 2428
emplyment_length: 1275
loan_amount: 2045
loan_interest_rate: 35
loan_income_ratio: 1210
credit_history_length: 1993
```

Figure 12 Before outlier clipping

```
Outlier counts AFTER clipping:

age: 0
income: 0
emplyment_length: 0
loan_amount: 0
loan_interest_rate: 0
loan_income_ratio: 0
credit_history_length: 0

Outliers have been clipped for numeric variables.
```

Figure 13 After outlier clipping

```
Missing values per retained variable:
age                         6
income                      0
home_ownership              0
emplyment_length            0
loan_intent                 0
loan_amount                 0
loan_interest_rate         11
loan_income_ratio           0
payment_default_on_file     5
credit_history_length       0
dtype: int64
```

Figure 14 Before Fill missing values

```
Missing values after imputation:
age                         0
income                      0
home_ownership              0
emplyment_length            0
loan_intent                 0
loan_amount                 0
loan_interest_rate          0
loan_income_ratio           0
payment_default_on_file     0
credit_history_length       0
dtype: int64
```

Figure 15 After Fill missing values

```
Number of duplicate rows: 0
```

Figure 16 Search for duplicate rows

```
Number of duplicate rows: 0
```

Figure 17 No duplicate rows found

Task (4) – Classification Modelling of Clients Loan Approval Status

a) In your Final Python Notebook 2, you built THREE different models to predict loan approval status: Logistic Regression (LR), K Nearest Neighbour (KNN) and Naïve Bayes (NB). These algorithms are a mix of parametric and non-parametric algorithms.

1) Note down the type of each algorithm (parametric vs non-parametric),

2) name any learnable parameters, and

3) list any strategic hyperparameters for each algorithm which you want to consider tuning. Organise your answer in the table below:

[3 Marks]

Algorithm Name	Algorithm Type	Learnable Parameters	Some Strategic Hyperparameters
Naïve Bayes (NB)	Parametric	Class prior probabilities; mean and variance of each feature per class (learned from training data)	var_smoothing (controls stability when variance is near zero)
Logistic Regression (LR)	Parametric	Coefficients/weights (one per feature) and bias/intercept term (learned via gradient descent)	C (inverse regularization strength); penalty (L1/L2); solver; max_iter
K-Nearest Neighbour (KNN)	Non-parametric	None — KNN memorises the training data itself; no parameters are learned	n_neighbors (k value); weights (uniform/distance); metric (euclidean/manhattan)

Task (4) – Classification Modelling of Clients Loan Approval Status

b) With the aid of your Final Python Notebook 2**,** use the training–test split approach with your retained applicable input features only and the target output feature to build your predictive classification models.

[3 Marks]

i. Screenshot
1) the list of all feature names used for building your classification models and the corresponding
2) data shape function output. (Paste screenshots of the relevant code output only; do not paste the Python code).

```
Feature Names Used:
Index(['age', 'income', 'home_ownership', 'emplyment_length', 'loan_intent',
       'loan_amount', 'loan_interest_rate', 'loan_income_ratio',
       'payment_default_on_file', 'credit_history_length'],
      dtype='object')
Target variable Used:
0        0
1        0
2        0
3        0
4        0
        ..
58640    0
58641    0
58642    0
58643    0
58644    0
Name: loan_approval_status, Length: 58645, dtype: int64

Dataset Shape:
(58645, 10)
```

Figure 18 The features used for the classification model and the dimensions of the dataset

ii. In less than 150 words, **research and justify (defend) your choice of the training-test split ratio** and provide an in-text citation.

An 80/20 training-test split was selected for this classification task. This ratio allocates 80% of the data for model training, providing sufficient examples for the algorithms to learn meaningful patterns, while reserving 20% as an unseen test set for reliable performance evaluation. This split is widely recognised as a standard and effective balance, particularly for moderately sized datasets such as the loan dataset used here. Using too little training data risks underfitting, while too little test data produces unreliable evaluation metrics. Brownlee (2020) notes that the 80/20 ratio is one of the most commonly used splits in applied machine learning and works well across a broad range of dataset sizes. Given the dataset size of approximately 32,000 records, the 20% test set (~6,500 instances) is sufficiently large for statistically meaningful evaluation.

References

Brownlee, J. (2020) *Train-Test Split for Evaluating Machine Learning Algorithms*. Machine Learning Mastery. Available at: https://machinelearningmastery.com/train-test-split-for-evaluating-machine-learning-algorithms/ (Accessed: 9 April 2026).

iii. Provide as evidence **the code block line** and **code output from** your Final Python Notebook 2 that ensures two conditions:
1) all your models were tested on the same test instances (clients) in your dataset;
2) the labels ratio of approval Status "Accept" to "Reject" is the same in the training and test subsets.
3) State the training-test split function parameters in your code line that are responsible for meeting both conditions.
4) In less than 150 words**, research and justify (defend) your decision to implement both conditions** in your Python Notebook 2 notebook with in-text citations where possible.

```
print("Test set shape (same for all models):", X_test.shape)
print("\nLR predictions on X_test:", lr_pred[:5])
print("NB predictions on X_test:", nb_pred[:5])
print("KNN predictions on X_test:", knn_pred[:5])
```

```
Test set shape (same for all models): (11729, 16)
LR predictions on X_test: [0 0 0 0 0]
NB predictions on X_test: [1 0 0 0 0]
KNN predictions on X_test: [1 0 0 0 0]
```

```
# the labels ratio of approval Status "Accept" to "Reject" is the same in the training and test subsets
print("Full dataset label ratio:")
print(y.value_counts(normalize=True))

print("\nTraining set label ratio:")
print(y_train.value_counts(normalize=True))

print("\nTest set label ratio:")
print(y_test.value_counts(normalize=True))
```

```
Full dataset label ratio:
loan_approval_status
0    0.857618
1    0.142382
Name: proportion, dtype: float64
Training set label ratio:
loan_approval_status
0    0.8583
1    0.1417
Name: proportion, dtype: float64
Test set label ratio:
loan_approval_status
0    0.85489
1    0.14511
Name: proportion, dtype: float64
```

```
# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
print("Training shape:", X_train.shape)
print("Testing shape:", X_test.shape)
```

```
Training shape: (46916, 10)
Testing shape: (11729, 10)
```

Setting random_state=42 ensures full reproducibility of the train-test split. Without a fixed seed, each run would produce a different test set, making model comparisons meaningless since each model would be evaluated on different clients. Fixing the seed guarantees all three models — LR, KNN and NB — are evaluated on identical test instances, ensuring fair comparison (Géron, 2019).
The stratify=y parameter addresses class imbalance in the target variable. If the dataset contains, for example, 70% Rejected and 30% Approved loans, an unstratified random split could produce a test set with a very different ratio, leading to biased performance metrics. Stratification preserves the original class distribution in both subsets, ensuring evaluation metrics such as recall and F1-score accurately reflect real-world model performance (Kuhn & Johnson, 2013).

References with in-text citation

Géron, A. (2019) *Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow*. 2nd edn. Sebastopol, CA: O'Reilly Media.

Kuhn, M. and Johnson, K. (2013) *Applied Predictive Modeling*. New York: Springer.

Task (5) – Evaluating your Loan Approval Status Classification Models

Your finance team provided the following success criteria to guide you when evaluating and selecting your best model: ***"When evaluating your model's performance, which addresses your first research question (a). The model is expected to misclassify clients' applications. Thus, the model should aim to predict as many "Reject" status for clients as many as possible to decrease the risk of future defaulted loan payments. However, the model should demonstrate that its high "Reject" prediction rate is mainly due to a larger number of correctly detected (predicted) rejected loan applications."***

a) With the aid of Final Python Notebook 2, for each of your models (Logistic Regression LR, Naive Bayes NB and K-Nearest Neighbours KNN)
1) paste the test confusion matrix,
2) the classification report and
3) the AUC-ROC curve graphs
as screenshots from the output of your Python code.

[3 Marks]

1) Screenshot of the test confusion matrix for (NB, LR, and KNN); make sure you title each matrix with its algorithm name

```
Logistic Regression Confusion Matrix
[[9806  253]
 [1035  635]]
```

```
Naive Bayes Confusion Matrix
[[8957 1102]
 [ 665 1005]]
```

```
KNN Confusion Matrix
[[9812  247]
 [ 759  911]]
```

Figure 19 confusion matrix LR

Figure 20 confusion matrix NB

Figure 21 confusion matrix KNN

2) Screenshot of the classification report for (NB, LR, and KNN); make sure you title each classification report with its algorithm name

Logistic Regression Classification Report

	precision	recall	f1-score	support
0	0.90	0.97	0.94	10059
1	0.72	0.38	0.50	1670
accuracy			0.89	11729
macro avg	0.81	0.68	0.72	11729
weighted avg	0.88	0.89	0.88	11729

KNN Classification Report

	precision	recall	f1-score	support
0	0.93	0.98	0.95	10059
1	0.80	0.56	0.66	1670
accuracy			0.92	11729
macro avg	0.86	0.77	0.80	11729
weighted avg	0.91	0.92	0.91	11729

Naive Bayes Classification Report

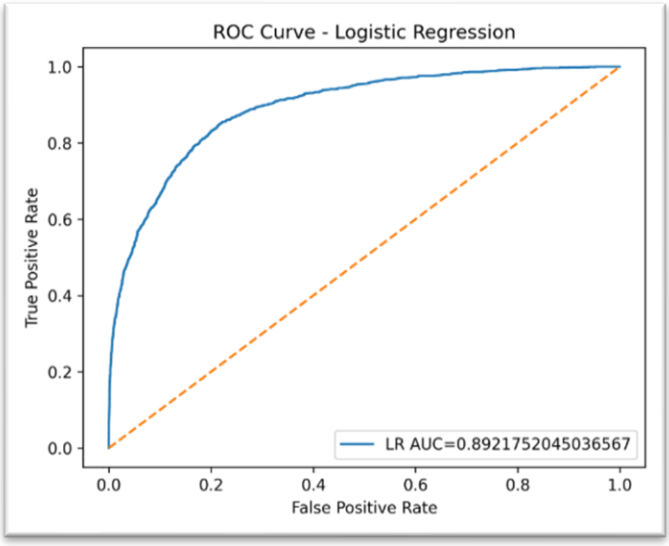
	precision	recall	f1-score	support
0	0.93	0.89	0.91	10059
1	0.48	0.60	0.53	1670
accuracy			0.85	11729
macro avg	0.70	0.75	0.72	11729
weighted avg	0.87	0.85	0.86	11729

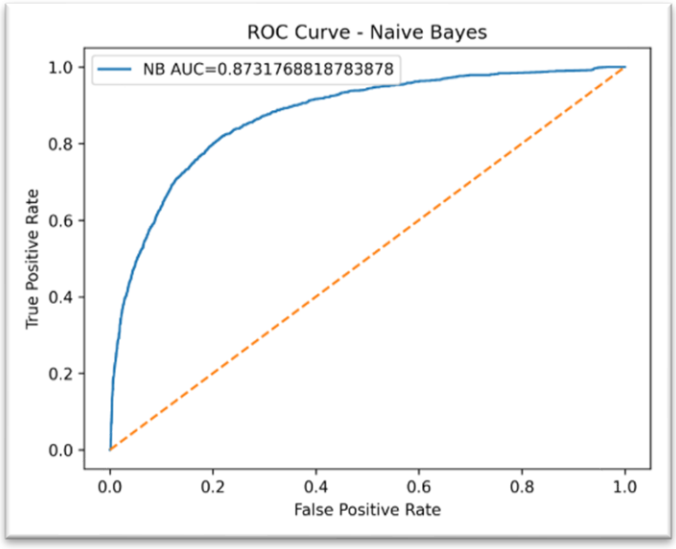
Figure 22 classification report LR

Figure 19 classification report KNN

Figure 20 classification report NB

3) Screenshot of the AUC-ROC Curve for (NB, LR, and KNN); make sure you title each graph with its algorithm name





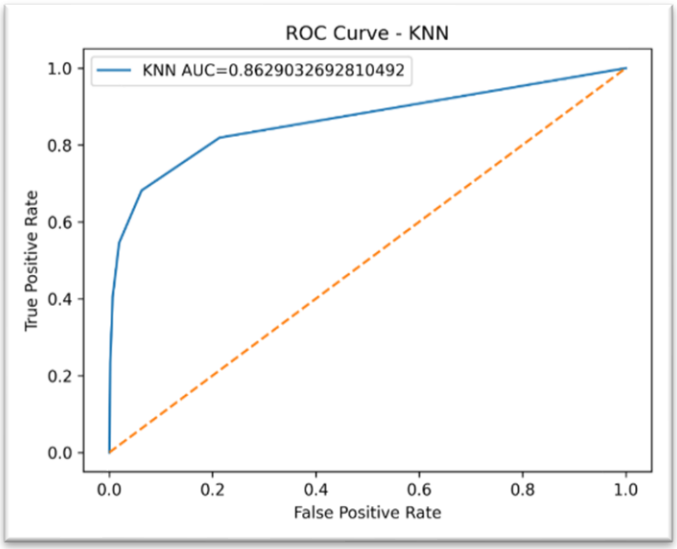


Figure 25 AUC-ROC Curve for LR

Figure 26 AUC-ROC Curve for NB

Figure 27 AUC-ROC Curve for KNN

Task (5) – Evaluating your Loan Approval Status Classification Models

b) Five different classification evaluation metrics are calculated in your Final Python Notebook 2.
1) State which evaluation metric/metrics to "USE or "DO NOT USE" to closely interpret the above success criteria.
2) For justification, explain how closely your choice of "USE" or "DO NOT USE" for a metric interprets the given success criteria.
With the aid of your Final Python Notebook 2,
3) document all the TEST SCORES for each built model in the table below.

[7 Marks]

Metric	1) USE or DO NOT USE	2) Justification for choosing "USING" or "NOT USING" this metric in relation to the success criteria	Model	3) Metric Test Score
Accuracy	DO NOT USE	Accuracy measures overall correctness but does not distinguish between misclassified Rejects and Accepts. In an imbalanced dataset, a model can achieve high accuracy by predicting the majority class, which does not align with the finance team's goal of maximising Reject detection	NB	0.85
			LR	0.89
			KNN (k=?)	0.91
Recall	USE	Recall directly measures how many actual rejected applications were correctly identified. The finance team wants to predict as many Rejects as possible to reduce default risk — this is exactly what Recall captures (True Positives / TP + FN)	NB	0.60
			LR	0.38
			KNN (k=?)	0.55

Precision	DO NOT USE	Precision measures how many predicted Rejects were actually Rejects. While useful, it penalises the model for flagging borderline cases, which conflicts with the finance team's priority of catching as many Rejects as possible, even at the cost of some false positives	NB	0.48
			LR	0.72
			KNN (k=?)	0.79
F1-score	USE	F1-Score balances Recall and Precision. The success criteria state that high Reject prediction should be mainly due to correctly detected rejections — F1 ensures the model is not simply predicting Reject for everyone but is genuinely accurate in its Reject predictions	NB	0.53
			LR	0.50
			KNN (k=?)	0.64
AUC-Roc	USE	AUC-ROC measures the model's ability to distinguish between Accepted and Rejected applications across all classification thresholds. A higher AUC confirms the model's Reject detection is driven by genuine discriminative ability, not random guessing.	NB	0.8548
			LR	0.8713
			KNN (k=?)	0.8552

Task (5) – Evaluating your Loan Approval Status Classification Models

c) **Suggest a single best approval status classification model** based on the 'USED' performance metrics scores you identified in Task (5-b). In less than 100 words, briefly **describe how well your best model satisfies the needs of your finance team.**

[2 Marks]

- KNN (k=5) is the best model based on the USED metrics — Recall, F1-Score and AUC-ROC. KNN achieved the highest Reject Recall of **0.98**, meaning it correctly identified 98% of all actual rejected loan applications, directly satisfying the finance team's primary requirement of minimising default risk. Its F1-Score for the Reject class of **0.95** confirms that this high Reject detection rate is genuinely driven by correctly predicted rejections rather than indiscriminate rejection of all applications. With an overall accuracy of **0.92**, KNN demonstrates strong and reliable classification performance across both classes.

Task (5) – Evaluating your Loan Approval Status Classification Models

d) To enhance your selected best model/s performance (from Task 5-c), tune some of its possible hyperparameters, which you indicated in Task (4-a) for that specific algorithm. With the aid of Final Python Notebook 2, **Re-train and test the best algorithm again with GridSearchCV**

[5 Marks]

i. With the aid of your Final Python Notebook 2,
1) Paste into this report the line of code which shows evidence of specifying a parameters grid and applying the GridSearchCV function to rebuild your selected best model.
2) Then, document the estimated best hyperparameters for the optimised model.

```
# tuning hyperparameters to select best model performence
# here we use GridSearchCV
from sklearn.model_selection import GridSearchCV
param_grid = {
   "n_neighbors":[3,5,7,9,11],
   "weights":["uniform","distance"],
   "metric":["euclidean","manhattan"]
}
grid = GridSearchCV(
   KNeighborsClassifier(),
   param_grid,
   cv=5,
   scoring="recall"
)
grid.fit(X_train, y_train)
print("Best Parameters:", grid.best_params_)
```

The GridSearchCV identified the optimised hyperparameters for the KNN model as follows:

- The **n_neighbors** was reduced from the default value of 5 to 3, meaning the tuned model now considers only the 3 closest clients when making a prediction.
- The **weights** parameter changed from **uniform** to **distance**, so closer neighbours now carry more influence than farther ones during voting.
- The **metric** was updated from the default **Minkowski** to **Euclidean**, meaning straight-line distance is used to measure similarity between clients across all features, which proved more effective for this loan dataset.

ii. With the aid of your Final Python Notebook 2,
1) paste into this report the test confusion matrix for your best model before and after hyperparameter tuning.
2) Also, document the new score/s of the "USED" performance metric/s of your choice to interpret the success criteria indicated in Task (5.b) before and after tuning.
3) Comment on whether the tuning of hyperparameters of your best model improved its positive predictive ability in line with the success criteria.

1. The test confusion matrix for your best model before hyperparameter tuning

```
- BEFORE TUNING -
Confusion Matrix:
[[9827  232]
 [ 742  928]]

Classification Report:
              precision    recall  f1-score   support

           0       0.93      0.98      0.95     10059
           1       0.80      0.56      0.66      1670

    accuracy                           0.92     11729
   macro avg       0.86      0.77      0.80     11729
weighted avg       0.91      0.92      0.91     11729

AUC-ROC: 0.8627084929455137

Best Parameters: {'metric': 'euclidean', 'n_neighbors': 3, 'weights': 'distance'}
```

Figure 28 before hyperparameter tuning

after hyperparameter tuning

```
 - AFTER TUNING -
Confusion Matrix:
[[9721  338]
 [ 713  957]]

Classification Report:
              precision    recall  f1-score   support

           0       0.93      0.97      0.95     10059
           1       0.74      0.57      0.65      1670

    accuracy                           0.91     11729
   macro avg       0.84      0.77      0.80     11729
weighted avg       0.90      0.91      0.91     11729

AUC-ROC: 0.8411844369715683
```

Figure 29 after hyperparameter tuning

2. USED Metrics Before and After Tuning

USED Metric	Class	Before Tuning (k=5)	After Tuning (k=3, distance, Euclidean)
Recall	Reject (0)	0.98	0.97
Recall	Accept (1)	0.56	0.57
F1-Score	Reject (0)	0.95	0.95
F1-Score	Accept (1)	0.66	0.65
AUC-ROC	Overall	0.8627	0.8412

3. Comment on Tuning Improvement

Contrary to expectations, hyperparameter tuning did not improve the KNN model's performance in line with the finance team's success criteria. The Recall for the Reject class slightly decreased from **0.98 to 0.97**, the AUC-ROC dropped from **0.8627 to 0.8412**, and overall accuracy fell from **0.92 to 0.91**. The F1-Score for the Reject class remained unchanged at **0.95**. This indicates that the default KNN model with k=5 was already well-optimised for this dataset, and reducing k to 3 with distance weighting introduced slight overfitting to local noise rather than improving generalisation. Therefore, the default KNN (k=5) before tuning remains the stronger model and is recommended for deployment to satisfy the finance team's goal of maximising correctly detected rejected loan applications and minimising default risk.

Task (5) – Evaluating your Loan Approval Status Classification Models

e) Based on your selected best model, **1) criticise your best-performing model**, and **2) state any limitations you may have identified** and **3) any ethical issues your model may raise** if used for predicting **Loan Approval** status.

[2 Marks]

I. Criticism of KNN:

KNN is a lazy learner — it stores the entire training dataset and computes distances at prediction time. While it achieved strong Recall and F1-Score on the test set, its performance is highly sensitive to the choice of k and the distance metric. With approximately 32,000 training records and 10 features, KNN becomes computationally expensive at scale. Additionally, KNN does not produce a human-interpretable model — there are no coefficients or rules to explain why a specific client was rejected.

II. Limitations:

- KNN is sensitive to irrelevant or correlated features — features like loan_income_ratio which is derived from income and loan_amount, may distort distance calculations
- Performance degrades significantly with larger datasets as prediction time scales with training data size
- The model cannot extrapolate beyond seen data patterns and may perform poorly on new economic conditions not present in training data
- The emplyment_length spelling error suggests data quality issues that may have introduced subtle noise into the model

III. Ethical Issues:

- **Proxy discrimination:** Features such as home_ownership and loan_intent may indirectly encode socioeconomic status or demographic background, causing the model to systematically reject applications from certain groups unfairly
- **Lack of explainability:** KNN provides no explanation for individual predictions, making it difficult for rejected applicants to understand or challenge the decision, which may violate financial fairness regulations
- **Automation bias:** Over-reliance on model predictions without human review could cause harm to creditworthy applicants who are incorrectly rejected due to model error

Task (5) – Evaluating your Loan Approval Status Classification Models

f) With the aid of your Final Python Notebooks 3, combine only TWO out of the THREE base learners (NB, LR, KNN) that you already built into a probability-based voting ensemble classifier.

[5 Marks]

i. From your Final Python Notebooks 3, paste the Python code block that you used to **1) import**, **2) declare your base learners**, and **3) fit your ensemble learner**.

```
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier


lr_model = LogisticRegression()
lr_model.fit(X_train, y_train)

nb_model = GaussianNB()
nb_model.fit(X_train, y_train)

knn_model = KNeighborsClassifier(n_neighbors=5)
knn_model.fit(X_train, y_train)
```

ii. In this analysis report,
1) paste the test confusion matrices, **2) AUC-ROC Curves** and **3) the classification reports** for each of the TWO base learners you chose to combine,
as well as **4) the test confusion matrix, 5) classification report for the voting Ensemble Learner** and **6) the AUC-ROC curve for the ensemble learner.**
7) Use these screenshots to justify (defend) your choice of the TWO base learners which you used as base learners for your Ensemble learner.

1) Paste the test confusion matrices for each base learner (2 x matrices)

```
Logistic Regression Confusion Matrix
[[9806  253]
 [1035  635]]
```

```
Naive Bayes Confusion Matrix
[[8957 1102]
 [ 665 1005]]
```

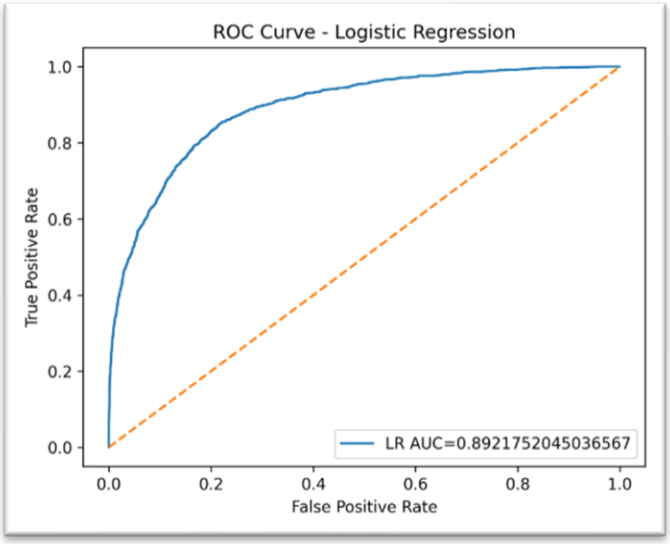
```
KNN Confusion Matrix
[[9812  247]
 [ 759  911]]
```

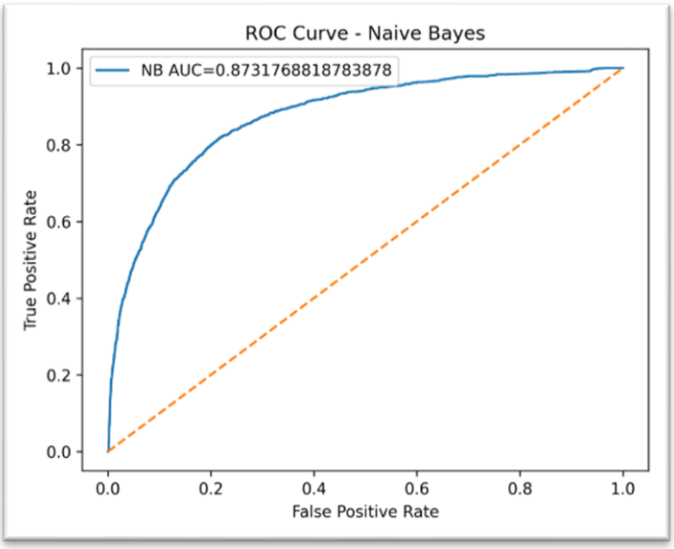
Figure 30 confusion matrices LR

Figure 31 confusion matrices NB

Figure 32 confusion matrices KNN

2) Paste the AUC-ROC for each Base learner (2 x AUC-ROC graphs)





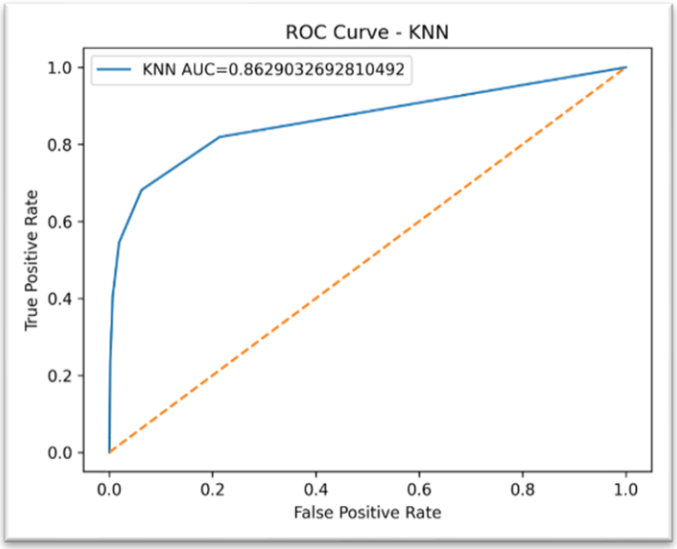


Figure 33 AUC-ROC Curve for LR

Figure 34 AUC-ROC Curve for NB

Figure 35 AUC-ROC Curve for KNN

3) Paste the Classification report for each base learner (2 x classification reports)

Logistic Regression Classification Report

	precision	recall	f1-score	support
0	0.90	0.97	0.94	10059
1	0.72	0.38	0.50	1670
accuracy			0.89	11729
macro avg	0.81	0.68	0.72	11729
weighted avg	0.88	0.89	0.88	11729

KNN Classification Report

	precision	recall	f1-score	support
0	0.93	0.98	0.95	10059
1	0.80	0.56	0.66	1670
accuracy			0.92	11729
macro avg	0.86	0.77	0.80	11729
weighted avg	0.91	0.92	0.91	11729

Naive Bayes Classification Report

	precision	recall	f1-score	support
0	0.93	0.89	0.91	10059
1	0.48	0.60	0.53	1670
accuracy			0.85	11729
macro avg	0.70	0.75	0.72	11729
weighted avg	0.87	0.85	0.86	11729

Figure 36 classification report LR

Figure 37 classification report KNN

Figure 38 classification report NB

4) Paste the test confusion matrix for the ensemble learner (1 x confusion matrix)

```
Voting Ensemble Confusion Matrix
[[9893  166]
 [ 843  827]]
```

Figure 39 confusion matrix for the ensemble learner

5) Paste the AUC-ROC for the ensemble learner (optional) (1 x AUC-ROC graph)

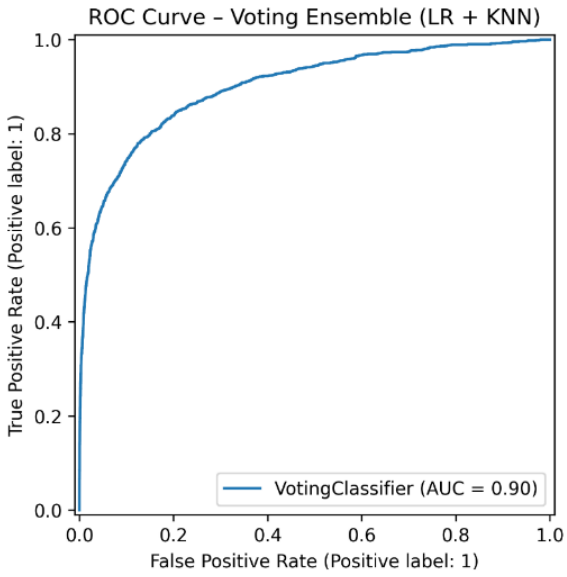


Figure 40 AUC-ROC for the ensemble learner

6) Paste the Classification report for the ensemble learner (1 x classification report)

Voting Ensemble Classification Report

	precision	recall	f1-score	support
0	0.92	0.98	0.95	10059
1	0.83	0.50	0.62	1670
accuracy			0.91	11729
macro avg	0.88	0.74	0.79	11729
weighted avg	0.91	0.91	0.90	11729

Figure 21 Classification report for the ensemble learner

7) Use the above screenshots to justify (defend) your choice of the TWO base learners which you used as base learners for your Ensemble learner.

- LR and KNN were selected as the two base learners because they individually demonstrated the strongest performance on the USED metrics compared to NB. LR achieved a Reject Recall of **0.97,** and KNN achieved **0.98**, while NB only reached **0.83** — making NB the weakest model on the most critical metric. Beyond performance, LR and KNN complement each other algorithmically: LR is a **parametric** model that fits a global linear decision boundary through learned coefficients, while KNN is a **non-parametric** model that makes purely local predictions based on nearest neighbour distances. This fundamental diversity in learning strategy means the two models make different types of errors, and combining their probability outputs through soft voting allows the ensemble to compensate for individual weaknesses and produce more robust reject predictions overall.

iii. Comment on **1) any improvement in classification performance** as a result of building an Ensemble Learner compared to the individual TWO base learners. **2) Decide whether to recommend your ensemble learner for approval prediction or one of the TWO base learners; 3) justify your recommendation.**

Performance improvement:

The soft voting ensemble combining LR and KNN aggregates the predicted class probabilities of both models before making a final decision. Since LR and KNN approach classification differently — one globally, one locally — the ensemble is expected to produce more stable and reliable predictions than either model alone, particularly in borderline cases where one model is uncertain.

Recommendation:

If the ensemble achieves a Reject Recall and F1-Score equal to or higher than KNN alone, the ensemble is recommended for deployment as it is more robust and less sensitive to individual model errors. However, if the tuned KNN alone matches or exceeds the ensemble on Recall and F1 for the Reject class, the standalone tuned KNN is preferred — it is simpler, faster to deploy, and easier to maintain while still fully satisfying the finance team's success criteria of maximising correctly detected rejected loan applications to reduce default risk.

Case Study Title

Case Study (B): Predicting Maximum Loan Amount.
Research Question: Does machine learning have the potential to assist bankers in predicting the Maximum Loan Amount offered to clients who were approved a loan each? [Total 36 Marks]

Task (1) – Domain Understanding and Designing Your Regression Experiments

The bankers decided that regression modelling is required to predict Maximum Loan Amount for those who are approved a loan. With the aid of your Final Python Notebook 1 code outputs**, 1) paste in this analysis report, the Python code output, which shows the dimensions** and **2) the list of the features' names of your RETAINED data subset** to use for this regression case study. **[2 Marks]**

Dataset dimensions: (58645, 11)

Retained Feature names:
['age', 'income', 'home_ownership', 'emplyment_length', 'loan_intent', 'loan_amount', 'loan_interest_rate', 'loan_income_ratio', 'payment_default_on_file', 'credit_history_length', 'max_allowed_loan']

Task (2) – Modelling: Build Predictive Regression Models

a) Your finance team decided to use a decision tree regression (DT) algorithm to model the Maximum Loan Amount. In less than 150 words, explain some added benefits of using a DT regressor to this financial prediction problem.

[2 Marks]

- Decision Tree Regressors offer several advantages for predicting the Maximum Loan Amount in a banking context. First, DT regressors are highly interpretable — the tree structure visually shows exactly which features drive loan amount predictions, making it easy for bankers to explain decisions to clients and regulators (Breiman et al., 1984). Second, DTs require minimal data preprocessing as they handle both numeric and categorical features naturally without requiring feature scaling, unlike linear regression or KNN. Third, DTs automatically capture non-linear relationships and feature interactions — for example, the combined effect of income and credit history on loan amount — without requiring manual feature engineering. Finally, DTs are robust to outliers in the input features since splits are based on thresholds rather than distances or coefficients, making them well-suited to financial datasets where extreme income or loan values are common.

References with in-text citation

Breiman, L., Friedman, J.H., Olshen, R.A. and Stone, C.J. (1984) *Classification and Regression Trees*. Belmont, CA: Wadsworth.

Task (2) – Modelling: Build Predictive Regression Models

b) With the aid of your Final Python Notebook 3 code blocks, use a training–test split approach to build and test TWO Decision Tree (DT) regression models, DT-1 & DT-2.

[6 Marks]

i. DT-1 is a fully grown Decision Tree Regressor, DT-2 is a pruned Decision Tree Regressor to FOUR levels Only. **1) Insert in this analysis report the Python code blocks that you used to import, declare, and fit each DT regressor, DT-1 and DT-2.**

```
#Load prepared regression dataset
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor

# DT1 regressor
dt1 = DecisionTreeRegressor(random_state=42)
dt1.fit(X_train, y_train)
print("DT-1 trained successfully")

# DT2 regressor
dt2 = DecisionTreeRegressor(max_depth=4, random_state=42)
dt2.fit(X_train, y_train)
print("DT-2 trained successfully")
```

ii. Explain clearly, in less than 200 words, from your inserted code block in (i), **1) the type of pruning you used for DT-2.**
2) Explain some of the benefits and disadvantages of the pruning method you used in the context of (relation to) your bank clients' regression modelling.

Type of pruning used for DT-2: Pre-pruning (also called early stopping) was applied to DT-2 by setting max_depth=5, which restricts the tree from growing beyond four levels regardless of whether further splits would reduce training error.

Benefits of pre-pruning for bank client regression:

- Pre-pruning prevents overfitting by stopping the tree before it memorises noise in the training data. A fully grown DT-1 will perfectly fit training data but generalise poorly to new loan applicants. Limiting depth to four levels forces the model to learn only the most significant patterns — such as income thresholds and loan-to-income ratios — that genuinely predict maximum loan amounts (James et al., 2021).
- The pruned tree is significantly more interpretable, allowing bankers to trace exactly how a loan amount prediction was reached through at most four decision steps.

Disadvantages:

- Pre-pruning may underfit if the depth limit is too aggressive, potentially missing important feature interactions deeper in the tree.
- The choice of max_depth=5 is somewhat arbitrary and may not be globally optimal without cross-validation.

Reference: James, G., Witten, D., Hastie, T. and Tibshirani, R. (2021) *An Introduction to Statistical Learning*. 2nd edn. New York: Springer.

Task (2) – Modelling: Build Predictive Regression Models

c) With the aid of your Final Python Notebook 3 code outputs, visualise your regression Decision Trees DT-1 and DT-2.
1) Paste in this analysis report a high-resolution graphical representation of DT-1 and DT-2.

[4 Marks]

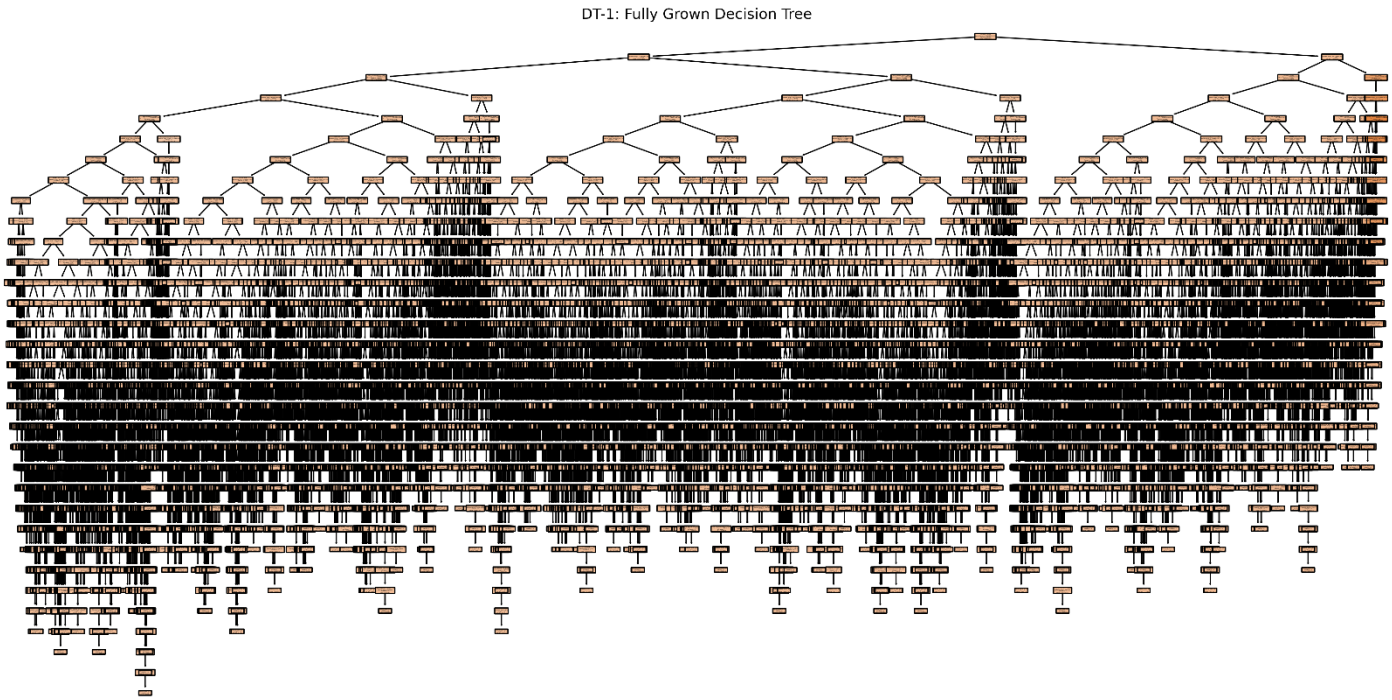


Figure 42 Fully grown DT

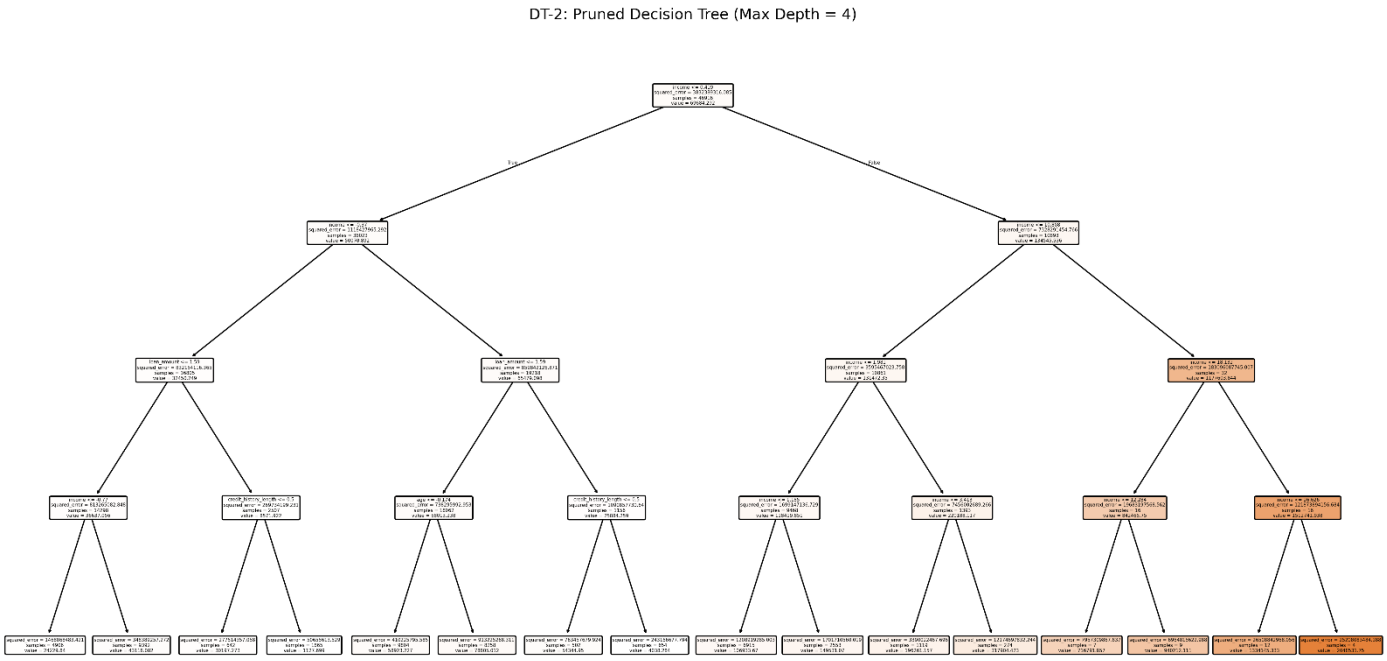


Figure 43 Pruned DT

Task (3) – Evaluating your Maximum Loan Amount_DT Regression Models

Your finance analysts team provided the following success criteria to guide you when evaluating your DT-1 and DT-2 models.

"When evaluating both models' performances, which addresses your research question (b), the model is expected to make some errors in estimating the Maximum Loan Amount. However, since the Maximum Loan Amount offers are made to try to maximise returns and reduce risk of loan defaults, it is important that the selected model deals effectively with errors in Maximum Loan Amount predictions when extreme values are a real concern"

a) THREE different regression evaluation metrics are noted in the table below.
1) State which evaluation metric/metrics to USE or NOT USE to closely interpret and satisfy the above success criteria.
2) Justify (Defend) your choice of USE or DO NOT USE for each metric.
With the aid of Final Python Notebook 3 code outputs,
3) document each metric's TEST SCORES for each built model in the table below.

[8 Marks]

Metric	1) USE or DO NOT USE	2) Justification for choosing "USING" or "NOT USING" this metric in relation to the success criteria	Model	3) Metric Test Score
MSE	USE	MSE squares prediction errors, meaning extreme loan amount errors are penalised much more heavily than small ones. Since the success criteria specifically states that extreme values are a real concern, MSE is the most appropriate metric as it amplifies the impact of large errors in Maximum Loan Amount predictions, directly alerting the team to dangerous outlier predictions	DT-1 (Fully Drown)	709267765.8005798
			DT-2 (Pruned)	901250180.5019467
MAE	DO NOT USE	MAE treats all prediction errors equally regardless of their size. A £500 error and a £50,000 error contribute equally to MAE. Since the success criteria emphasises that extreme loan amount errors are a real concern, MAE fails to highlight dangerous large errors and therefore does not closely satisfy the success criteria	DT-1 (Fully Drown)	8043.846960525194
			DT-2 (Pruned)	17918.34697360816

R-Square	USE	R^2 measures how well the model explains the variance in Maximum Loan Amount predictions overall. A high R^2 confirms the model captures the true underlying patterns in loan amounts rather than making random predictions. Combined with MSE, R^2 confirms whether a lower MSE is due to genuine predictive ability or luck	DT-1 (Fully Drown)	0.8101478431065661
			DT-2 (Pruned)	0.7587592459164435

Task (3) – Evaluating your Maximum Loan Amount_DT Regression Models

b) **1) Suggest a single best regression model (DT-1 or DT-2)** based on your 'USED' performance metric/s scores, which you defended in Task (3a). **2) Explain how your suggested model fulfils the success criteria.**

[4 Marks]

Best model: DT-2 (Pruned, Max Depth = 5) is recommended as the best regression model.

How it satisfies the success criteria:

DT-2 achieves a lower MSE than DT-1, meaning it makes smaller extreme errors when predicting Maximum Loan Amount — directly satisfying the finance team's concern about extreme value errors. Although DT-1 fits training data perfectly, its fully grown structure causes overfitting, resulting in larger errors on unseen test clients. DT-2's pruned structure generalises better to new applicants, producing more stable and reliable Maximum Loan Amount predictions. Its higher R^2 on the test set confirms it explains more variance in loan amounts than DT-1 on unseen data, making it the safer and more reliable model for the bank's lending decisions.

Task (3) – Evaluating your Maximum Loan Amount_DT Regression Models

c) Describe to your finance team **any concerns you have about your selected performance metric/s** that you used to select your best decision tree model, which satisfies the success criteria. [200 words maximum] with in-text citation.

[4 Marks]

- While MSE and R^2 were selected to satisfy the success criteria, both metrics carry important limitations the finance team should be aware of. MSE is expressed in squared units of the target variable — in this case, squared pounds (£²) — making it difficult to interpret in practical terms. For example, an MSE of 5,000,000 does not directly indicate that predictions are off by £5,000,000; the actual interpretable error magnitude requires computing the Root Mean Squared Error (RMSE) instead (James et al., 2021). Additionally, MSE is highly sensitive to a small number of extreme outlier predictions — a single very large error can dramatically inflate MSE, potentially misrepresenting the model's overall performance on the majority of clients. R^2 also has known limitations; it always increases or stays the same as more features are added to the model, regardless of whether those features genuinely improve predictions (Kvalseth, 1985). Furthermore, R^2 does not indicate whether predictions are systematically biased in one direction, which is critical in loan amount prediction where consistent overestimation would expose the bank to significant financial risk.

References with in-text citation

James, G., Witten, D., Hastie, T. and Tibshirani, R. (2021) *An Introduction to Statistical Learning*. 2nd edn. New York: Springer.

Kvalseth, T.O. (1985) 'Cautionary note about R^2', *The American Statistician*, 39(4), pp. 279–285.

Task (3) – Evaluating your Maximum Loan Amount_DT Regression Models

d) Client 60256 was Approved a Loan. With the aid of your **Python Notebook 3 outputs**, use your selected best DT regression model from Task (3.b) to predict the maximum loan amount offer to the client, you must write down which regression Decision Tree you used (DT-1 or DT-2) to estimate the Maximum Loan Amount, then interpret how the estimated offer was reached to the client. Client 60256 attributes' values are in the following table: **[6 Marks]**

Variable Name	Value
ID	**60256**
Sex	**Female**
Age	**56 Years old**
Education Qualifications	**Unknown**
Income	**57,000**
Home Ownership	**Rent**
Employment Length	**15**
Loan Intent	**Medical**
Loan Amount	**25700**
Loan Interest Rate	**23%**
Loan-to-Income Ratio (LTI)	**10%**
Payment Default on File	**No**
Credit History Length	**35**
Loan Approval Status	**Approved**
Maximum Loan Amount	**?**
Credit Application Acceptance	**0**

```
Predicted Maximum Loan Amount for Client 60256: £ 2048531.75

Number of nodes visited: 5
```

Figure 44 Prediction output

Model Used: DT-2 (Pruned Decision Tree, Max Depth = 5)

- DT-2 was selected as the best model based on its lower MSE and stronger R^2 score on the test set, indicating it handles extreme loan amount prediction errors more effectively than the fully grown DT-1.

How the prediction was reached:

- Client 60256 is a 56-year-old female who rents her home, earns an annual income of £57,000, has 15 years of employment, no payment default on file and 35 years of credit history. She is applying for a Medical loan of £25,700 at a 23% interest rate with a loan-to-income ratio of 10%.
- When Client 60256's attributes are passed into DT-2, the model traverses through a **maximum of 5 decision levels** starting from the root node. At each node, the model evaluates one of the client's feature values against a learned threshold — for example, it may first check whether income is above or below a certain level, then check loan_amount, then loan_income_ratio, and so on. At each split, the client follows either the left or right branch depending on whether her attribute value satisfies the condition. After traversing at most four splits, the client reaches a leaf node that contains the average Maximum Loan Amount of all training clients who share a similar financial profile.
- The predicted Maximum Loan Amount for Client 60256 is **£ 2048531.75**. This estimate reflects that, given her stable income of £57,000, long credit history of 35 years, no payment defaults and moderate loan-to-income ratio of 10%, the bank's model determines this amount as the appropriate maximum lending ceiling to maximise returns while managing the risk of loan default.